

Ricorsione

Sunday, April 26, 2026

5:08 PM

La ricorsione può essere diretta o indiretta. Necessita in entrambi i casi di una condizione di terminazione.

PARADIGMA DIVIDE ET IMPERA

- Divide: da problema di dimensione n in $a \geq 1$, problemi indipendenti e di uguale natura di dimensione $n' < n$.
- Impera: risoluzione di un problema elementare (condizione di terminazione).
- Combina: ricostruzione di soluzione complessiva combinando le soluzioni parziali.

Ne esistono di due tipologie a seconda del valore di n' :

- Divide and conquer: fattore di riduzione costante
- Decrease and conquer: valore di riduzione costante.

Anche il calcolo della complessità varia leggermente e viene effettuato mediante l'equazione alle ricorrenze. $T(n)$ viene espresso in termini di:

- $D(n)$: costo della divisione
- Tempo di esecuzione per input più piccoli (ricorsione)
- $C(n)$: costo della ricombinazione

Supponendo il costo della terminazione unitario.

Se:

- a : numero di sottoproblemi nella fase di Divide
- b : fattore di riduzione, quindi n/b e la dimensione di ciascun sottoproblema.

L'equazione alle ricorrenze diventa:

$$T(n) = D(n) + a T\left(\frac{n}{b}\right) + C(n) \quad n > c$$

$$T(n) = \Theta(1) \quad n \leq c$$

Se:

- a : numero di sottoproblemi nella fase di Divide
- La riduzione è di un valore k che può variare di passo in passo.

L'equazione alle ricorrenze diventa:

$$T(n) = D(n) + \sum_{i=0}^{a-1} T(n - k_i) + C(n) \quad n > c$$

$$T(n) = \Theta(1) \quad n \leq c$$

Una funzione si dice tail-recursive se la chiamata ricorsiva è l'ultima operazione da eseguire, eccezion fatta per la return. In questo caso i calcoli si fanno in fase di discesa.

```
Return tr_fcat(n-1, n*f);
```

Questa è tail-recursive perché la moltiplicazione viene eseguita prima della chiamata ricorsiva. La funzione chiamante deve solo ritornare il valore calcolato dalla funzione chiamata. L'operazione di pop dallo stack frame della funzione chiamante avviene prima dell'operazione di push nello stack frame della funzione chiamata. Per cui l'occupazione di memoria non è $O(n)$ dove n sono i livelli di ricorsione ma $O(1)$, per cui non si può verificare stack overflow.

```
Return n*fact(n-1);
```

Non è tail-recursive poiché la moltiplicazione può essere ritornata solo dopo il ritorno della chiamata ricorsiva. Per cui i calcoli si effettuano in fase di salita.

Per cui se la funzione è tail-recursive si può trasformare in iterativa senza uso di stack, altrimenti solo con l'uso dello stack.

ORDINAMENTI RICORSIVI

1) Merge sort

Divisione: due sottovettori SX e DX rispetto al centro del vettore.

Ricorsione:

- Condizione di terminazione: con 1 o 0 elementi il vettore è ordinato
- Merge sort su sottovettore DX
- Merge sort su sottovettore SX

Ricombinazione: fonde i due sottovettori ordinati in un vettore ordinato.

- 2-Way Merge: La dimensione del vettore A è una potenza di 2.
 - o Fusione di 2 sottovettori di A ordinati di uguale dimensione per ottenere un vettore ordinato di dimensione doppia
 - o Generalizzabile a k vettori
 - o Indice q per dividere i sottovettori di A a metà in due sottovettori sinistro e destro
 - o Scorrere i sottovettori sinistro e destro mediante gli indici i e j e il vettore B mediante k.
 - o Se è esaurito il sottovettore sinistro, ricopiare gli elementi nel sottovettore destro di B
 - o Se è esaurito il sottovettore destro, ricopiare gli elementi nel sottovettore sinistro di A
 - o Confrontare l'elemento A[i] con A[j]
 - Se $A[i] \leq A[j]$, copiare A[i] in B e incrementare i, j resta invariato
 - Altrimenti, copiarlo in B e incrementare j, i resta invariato.

```
Void MergeSortR(int A[], int B[], int l, int r){
    Int q;
    If(l >= r){
        Return;
    }
    Q = (l+r)/2;
    MergeSortR(A, B, l, q);
    MergeSortR(A, B, q+1, r);
    Merge(A, B, l, q, r);
}
```

```
Void Merge(int A[], int B[], int l, int r, int q){
    Int i, j, k;
    i=l;
    j=q+1;
    For(k=l; k<=r; k++){
        If(i>q)
            B[k] = A[j++];
        Else if(j>r)
            B[k] = A[i++];
        Else if((A[i] < A[j]) || (A[i] == A[j]))
            B[k] = A[j++];
        Else
            B[k] = A[i++];
    }
    For(k=l; k<=r; k++){
        A[k] = B[k];
    }
}
```

2) Quick Sort

Divisione:

- Partiziona A in due sottovettori Sx e Dx:
 - o Dato un pivot $x=A[q]$
 - o SX contiene tutti gli elementi $\leq x$

- DX contiene tutti gli elementi $\geq x$

La divisione non è necessariamente a metà come nel caso del merge sort.

Ricorsione:

- Quicksort su SX
- Quicksort su DX
- Condizione di terminazione: se il vettore ha 1 elemento

No ricombinazione.

- Partition a la Hoare
 - Pivot $x = A[r]$
 - I per scandire da sx a dx e j da dx a sx.
 - Ripeti finché $i < j$
 - Individua $A[i]$ e $A[j]$
 - Ciclo ascendente su I fino a trovare un elemento maggiore di x
 - Ciclo discendente su j fino a trovare un elemento minore di x
 - Scambia $A[i]$ e $A[j]$
 - Alla fine scambia $A[i]$ con x
- Ritorna $q = i$.

Esistono varie partition.

```
Void QuickSort(int A[], int N){
    Int l=0, r=N-1;
    quickSortR(A, l, r);
}
```

```
Void quickSortR(int A[], int l, int r){
    Int q;
    If (l>=r)
        Return;
    Q = partition(A,l, r);
    quickSortR(A, l, q-1);
    quickSortR(A, q+1, r);
}
```

```
Int partition(int A[], int l, int r){
    Int I = l-1, j=r;
    Int x =A[r];
    For( ; ; ){
        While(A[++i] <x);
        While(A[--j] > x);
        If(I >= j)
            Break;
        Swap(A, I, j);
    }
    Swap(A, I, r);
    Return I;
}
```